COMPUTING LAB - II
ADVANCED SYSTEM SOFTWARE LAB - CSE 5261
MTech - CSE
Week - 2 Basics of Python Programming

Name : Rajat Jain
Reg No : 200913010

Lab Exercises:

1) Write a python program that -
a) creates the list [1,11,111,1111,...,111...1], where the entries have an ever increasing number of ones, with the last entry having 100 ones.

Code

```
ones = []
for i in range(1,101):
    ones.append(int('1'*i))
print(ones)
```

Output

IDLE Shell 3.9.2
File Edit Shell Debug Options Window Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemSoftware\Lab\Lab2\prog1a.py
Squeezed text (66 lines).
>>>
========== RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemSoftware\Lab\Lab2\prog1a.py ==========
[1, 11, 111, 1111, 11111, 111111, 1111111, 11111111, 111111111, 1111111111, 11111111111, 111111111111, 1111111111111, 11111111111111, 111111111111111,
11111111111111111

b) finds all pairs of six-digit palindromic numbers that are less than 20 apart. One such pair is 199991 and 200002.

<u>Code</u>

```
pali=[]

def isPalindrome(s):
    return s == s[::-1]


# Driver code
for i in range(100000,1000000):
    ans = isPalindrome(str(i))

    if ans:
       pali.append(int(i))
    else:
        pass


for i in range(0,len(pali)):

    j=i+1
    while(j!=len(pali)):
        if(pali[j]!=pali[i]):
            if (pali[j] -pali[i] < 20) :

                    print(pali[i],pali[j])
        j+=1
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog1b.py
199991 200002
299992 300003
399993 400004
499994 500005
599995 600006
699996 700007
799997 800008
899998 900009
>>> |
```

c) asks the user to enter a date in the format mm/dd/yy and converts it to a more verbose format. For example, 02/04/77 should get converted into February 4,
1977.

Code

```
from datetime import datetime
date_input = input('Enter a date(mm/dd/yyyy): ')
date_object = datetime.strptime(date_input, '%m/%d/%Y')
print(date_object.strftime('%B %d, %Y'))
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog1c.py
Enter a date(mm/dd/yyyy): 05/20/1994
May 20, 1994
>>> |
```

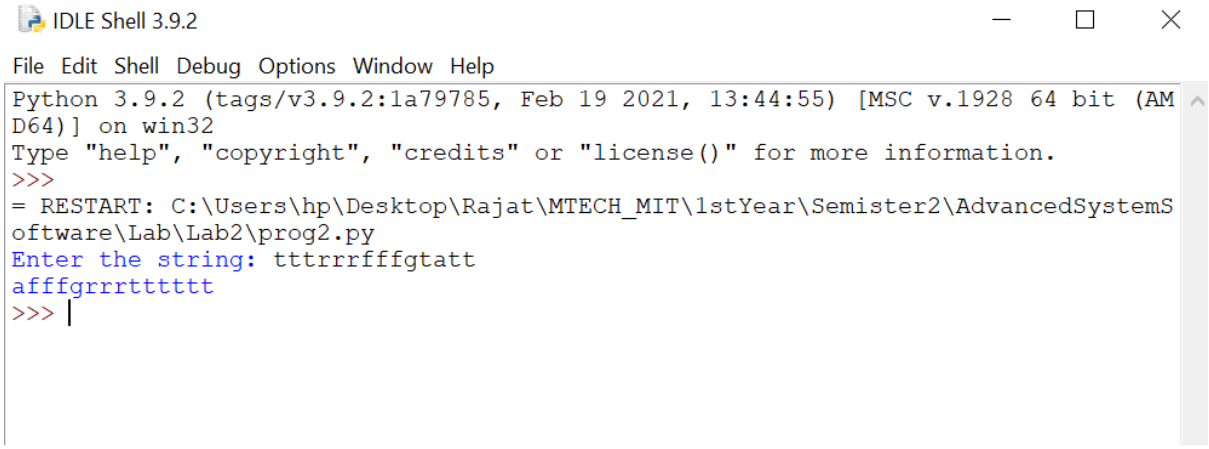
2) Write a program that asks the user to enter a word. Rearrange all the letters of
the word in alphabetical order and print out the resulting word. For example, abracadabra should become aaaaabbcdrr.

Code

```
def rearrange(str):
   return ''.join(sorted(str))

str = input("Enter the string: ")
print(rearrange(str))
```

Output



3) Create a 5 × 5 list of numbers. Then write a program that creates a dictionary whose keys are the numbers and whose values are the how many times the number occurs. Then print the three most common numbers

Code

```
import random
import itertools
import operator
five_dim = [[random.randint(1,5) for b in range(0,5)]for a in range(0,5)]
flat_list = [num for num in itertools.chain(*five_dim)]
key_set = set(flat_list)
freq_dict ={k: flat_list.count(k) for k in key_set}
x= sorted(freq_dict.items(), key = operator.itemgetter(1))[-3:]
print(f"The 5*5 list list is {five_dim}")
```

```
print(f"The dictionary is {freq_dict}")
print("The three most common numbers are : ")
for i in x:
    print(i[0])
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog3.py
The 5*5 list list is [[3, 5, 2, 3, 2], [5, 2, 1, 2, 4], [5, 3, 2, 5, 3], [2, 5,
1, 2, 3], [4, 4, 2, 1, 4]]
The dictionary is {1: 3, 2: 8, 3: 5, 4: 4, 5: 5}
The three most common numbers are :
3
5
2
>>>
```

4) Write a Python program that uses a dictionary that contains ten user names and passwords. The program should ask the user to enter their username and password. If the username is not in the dictionary, the program should indicate that the person is not a valid user of the system. If the username is in the dictionary, but the user does not enter the right password, the program should say that the password is invalid. If the password is correct, then the program should tell the user that they are now logged in to the system.

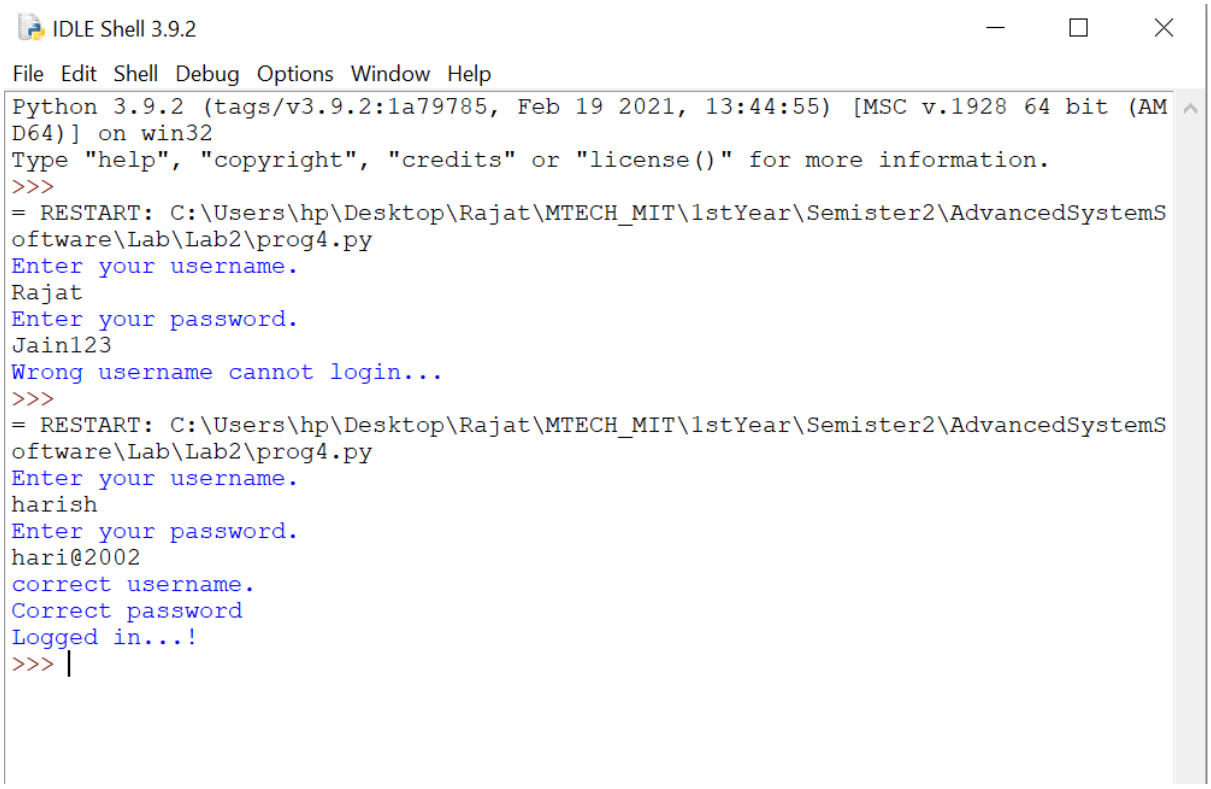
Code

```
name_pass={"harish":"hari@2002","raghu":"r123@ghu","shri":"shri@98","rite
sh":"ritz@2001","karthik":"kar@904","rajat":"rajat@1223","ashwin":"ash@19
98","manisha":"manisha@1289","aditya":"desai@1992","anusha":"anusha@2
002"}

login_name = str(input("Enter your username.\n"))
login_pass = str(input("Enter your password.\n"))
if login_name in name_pass.keys():
    print("correct username.")
    if login_pass == name_pass[login_name]:
        print("Correct password")
        print ("Logged in...!")
```

```
    else:
        print("Wrong password cannot login...")
else:
    print("Wrong username cannot login...")
```

<u>Output</u>



5) Suppose you are given the following list of strings:
L = ['aabaabac', 'cabaabca', 'aaabbcba', 'aabacbab', 'acababba']
Patterns like this show up in many places, including DNA sequencing. The user has a string of their own with only some letters filled in and the rest as asterisks. An example is a**a****. The user would like to know which of the strings in the list fit with their pattern. In the example just given, the matching strings are the first and fourth. Write a program to find the strings that match a user-entered string.

<u>Code</u>

```
s = input("Enter the string")
match_index = []
L = ['aabaabac', 'cabaabca', 'aaabbcba', 'aabacbab', 'acababba']
```

```
for i in range(len(L)):
    match = True
    for j in range(len(s)):
        if s[j]!='*' and s[j]!=L[i][j]:
            match = False
    if(match):
        match_index.append(i+1)
print("The strings in the list which match the user string are : ")
for x in match_index:
    print(x)
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog5.py
Enter the stringa**a****
The strings in the list which match the user string are :
1
4
>>> |
```

6) You are given a file namelist.txt that contains a bunch of names. Some of the names are a first name and a last name separated by spaces, like George Washington, while others have a middle name, like John Quincy Adams. There are no names consisting of just one word or more than three words. Write a program that asks the user to enter initials, like GW or JQA, and prints all the names that match those initials. Note that initials like JA should match both John Adams and John Quincy Adams.

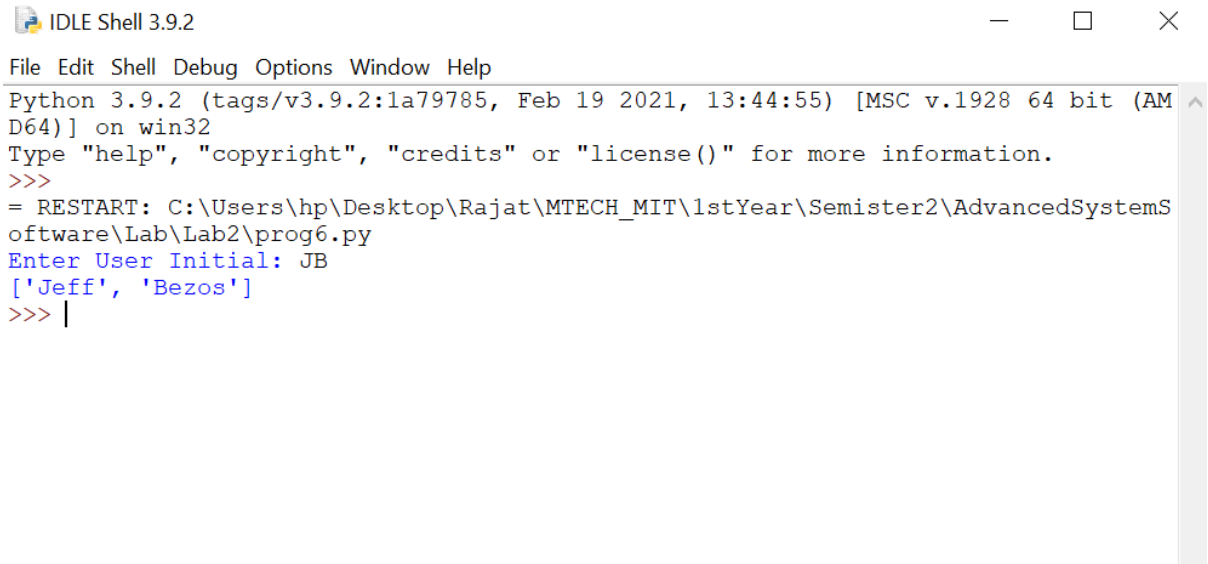
Code

```
with open("namelist.txt") as n:
    s = n.readlines()
lines = [line.strip() for line in s]
str_data = [line.split(" ") for line in lines]
Init = (input("Enter User Initial: "))
initial = list(Init)

for list in str_data:
```

```
    if len(initial)>=2:
        if len(list)==3:
            if initial[0] == list[0][:1] and initial[1] == list[1][:1] or initial[1] ==
list[2][:1]:
                print(list)
        elif len(list)==2:
            if initial[0] == list[0][:1] and initial[1] == list[1][:1]:
                print(list)
    else :
        print("Enter either two or three initials")
        break
```

Output



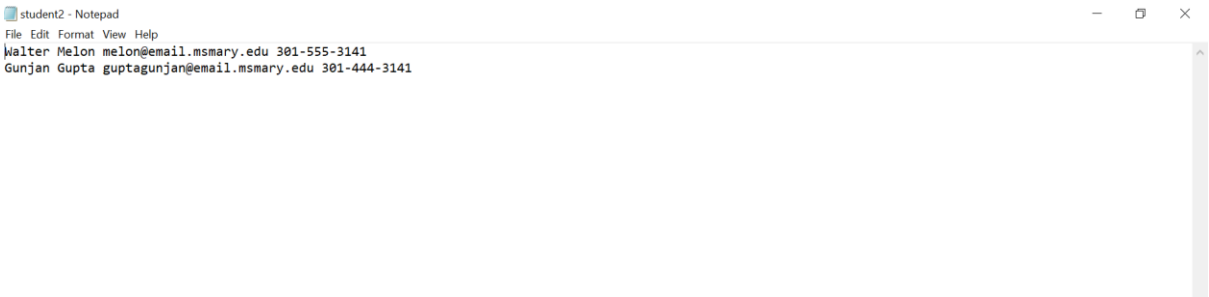
7) You are given a file called students.txt. A typical line in the file looks like:
Walter melon melon@email.msmary.edu 555-3141
There is a name, an email address, and a phone number, each separated by tabs. Write a Python program that reads through the file line-by-line, and for each line, capitalizes the first letter of the first and last name and adds the area code 301 to the phone number. Your program should write this to a new file called students2.txt. Here is what the first line of the new file should look like:
Walter Melon melon@email.msmary.edu 301-555-3141
CSE 5261 Page 2 of 2

Code

```
file=open("student.txt",'r')
file2=open("student2.txt",'w')
```

```
l=[]
for i in file:
    r=i.split()
    file2.write(r[0].capitalize()+" "+r[1].capitalize()+" "+r[2]+" "+"301-
"+r[3]+"\n")
file.close()
file2.close()
```

<u>Output</u>



8) Write a function called closest that takes a list of numbers L and a number n and returns the largest element in L that is not larger than n. For instance, if L=[1,6,3,9,11] and n=8,then the function should return 6, because 6 is the closest thing in L to 8 that is not larger than 8.

<u>Code</u>

```
def closest(l,n):
    x = [b for b in l if n>b]
    return max(x)
res = closest([1,6,3,9,11],8)
print(res)
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog8.py
6
>>> |
```

9) The digital root of a number n is obtained as follows: Add up the digits n to get a new number. Add up the digits of that to get another new number. Keep doing this until you get a number that has only one digit. That number is the digital root. For example, if n = 45893, we add up the digits to get 4 + 5 + 8 + 9 + 3 = 29. We then add up the digits of 29 to get 2 + 9 = 11. We then add up the digits of 11 to get 1 + 1 = 2. Since 2 has only one digit, 2 is our digital root. Write a function that returns the digital root of an integer n. [Note: there is a shortcut, where the digital root is equal to n mod 9, but do not use that here.]

Code

```
num = input("Ener the number")
num1 = num
while int(num)>9:
    sum = 0
    for digit in str(num):
        sum+=int(digit)
    num=sum
print(f"The digital root of the number {num1} is {sum}")
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog9.py
Ener the number45893
The digital root of the number 45893 is 2
>>> |
```

10) Write a function called first_diff that is given two strings and returns the first location in which the strings differ. If the strings are identical, it should return -1.

Code

```
def first_diff(s1,s2):
    if s1==s2:
        return -1
    else:
        l1 = len(s1)
        l2 = len(s2)
        for i in range(min(l1,l2)):
            if s1[i]!=s2[i]:
                return(i)
location = first_diff('Rajat','Rajeev')
if location==-1:
    print("The two strings are same")
else:
    print(f"The two strings first differ in the location {location}")
```

Output

```
IDLE Shell 3.9.2
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AM
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\hp\Desktop\Rajat\MTECH_MIT\1stYear\Semister2\AdvancedSystemS
oftware\Lab\Lab2\prog10.py
The two strings first differ in the location 3
>>>
```